

CSC240 Enriched Introduction to the Theory of Computation

Brian Zu

2026 Winter

Contents

Chapter 1	What's a proof	Page 3
Chapter 2	What is a proof?	Page 4
2.1	Propositions	4
2.2	Predicate Logic	5
2.3	Proof Techniques	8
2.4	Proof of Equivalences and Advanced Proof Techniques	11
2.5	Recursively defined sets	14
Chapter 3	Well Ordered Principles	Page 21
3.1	Countable and Uncountable Sets	22
3.2	Merge Sort	25
	Example 2: Multiplication — 28	
3.3	Finite Automata	31
	Recursive Definition of δ^* — 33	
3.4	NFA and DFA Have the Same Power	33
Chapter 4	Regular Languages and the Pumping Lemma	Page 37
4.1	Regular Languages	37
4.2	Non-Regular Languages	37
4.3	The Pumping Lemma	38
4.4	Application: $T = \{a^m b^n \mid m \neq n\}$ is Not Regular	38
Chapter 5	Closure Properties of Regular Languages	Page 40
5.1	Closure Results	40
	Proof of Closure under Complement — 40 • Proof of Closure under Union — 40 • Closure under Intersection — 42 • Closure under Set Difference — 42	

5.2	Closure under Concatenation and Kleene Star	42
	PREFIX Construction — 43	
5.3	Second Proof that $T = \{a^m b^n \mid m \neq n\}$ is Not Regular	44

Chapter 6 Regular Expressions Page 45

6.1	Definition of Regular Expressions	45
	Useful Identities — 46 • Equivalence of Regular Expressions — 47 • Generalized Regular Expressions — 47	
6.2	Regular Expressions and Finite Automata	47
	From Regular Expressions to Finite Automata — 48 • From Finite Automata to Regular Expressions — 49	
6.3	General Transition Graphs	49
	NFA to GTG Conversion — 50 • GTG to Regular Expression (State Elimination) — 51 • Example: NFA to Regular Expression via State Elimination — 53	

Chapter 1

What's a proof

Chapter 2

What is a proof?

2.1 Propositions

Definition 2.1.1: Proposition

A proposition is a statement that either true or false.

Example 2.1.1

"For every non-negative integer n , $n^2 + n + 41$ is prime." This is a proposition that's false.

Definition 2.1.2: Disjunction

The **disjunction** of P and Q is denoted $P \text{ OR } Q$.

Definition 2.1.3: Conjunction

The **conjunction** of P and Q is denoted $P \text{ AND } Q$.

Definition 2.1.4: Exclusive-or

The exclusive or of P and Q is denoted $P \text{ XOR } Q$. It can be expanded as

$$P \text{ XOR } Q = (\text{NOT}(P) \text{ AND } Q) \text{ OR } (P \text{ AND } \text{NOT}(Q))$$

Definition 2.1.5: Implies

Implication is denoted by $P \text{ IMPLIES } Q$, where P is **hypothesis** and Q is **conclusion**.

It's logically equivalent to

$$\text{NOT}(P) \text{ OR } Q$$

Definition 2.1.6: Contrapositive

The contrapositive of $P \text{ IMPLIES } Q$ is $\text{NOT}(Q) \text{ IMPLIES } \text{NOT}(P)$

Definition 2.1.7: Converse

The converse of $P \text{ IMPLIES } Q$ is $Q \text{ IMPLIES } P$.

Definition 2.1.8: IFF

If and only if is denoted by $P \text{ IFF } Q$.

$$P \text{ IFF } Q = \text{NOT}((\text{NOT}(P) \text{ AND } Q) \text{ OR } (P \text{ AND } \text{NOT}(Q)))$$

Theorem 2.1.1

Every integer greater than one is a product of primes. Let $M = \{n \in \mathbb{Z} | n \geq 1\}$. For all $m \in M$, let $p(m) = "m \text{ is a product of primes}"$.

Proof: Let $n \in M$ be arbitrary. Suppose $\forall i \in M((i < n) \text{ IMPLIES } p(i))$. If n is prime, then n is a product of 1 prime, so $p(n)$. Otherwise, by definition of prime, $\exists k \in M \exists m \in M.(n = m \cdot k)$. Since $k < n, m < n$, it follows by the induction hypothesis that k is a product of primes and m is a product of primes. Thus $n = km$ is a product of primes, so $p(n)$. By strong induction, $\forall n \in M.p(n)$ ☺

2.2 Predicate Logic

Definition 2.2.1: Predicate Logic Formula

A predicate logic formula is an expression built from:

- predicate symbols (each with fixed arity),
- variables and constant symbols from specified domains,
- logical connectives,
- universal ($\forall$) and existential ($\exists$) quantifiers.

Example 2.2.1

Example:

$$\forall x \in D.(P(x, y, 0) \text{ IMPLIES } \exists y \in D.(S(x, y) \text{ AND } G(y)))$$

A constant symbol denotes one fixed element of the domain, while a variable may denote any element of the domain.

Definition 2.2.2: Free and Quantified Variables

An occurrence of a variable x is **quantified** if it occurs inside a subformula of the form

$$\forall x \in D.E \quad \text{or} \quad \exists x \in D.E.$$

Otherwise, the occurrence is **free**.

Constants are never quantified.

Important:

- Do not use the same symbol for a free variable and a quantified variable.
- Do not reuse the same symbol for nested quantified variables.

Definition 2.2.3: Interpretation

An interpretation of a predicate logic formula consists of:

1. A nonempty set for each domain.
2. A function for each function symbol.
3. A truth-valued function for each predicate symbol.
4. A domain element for each constant symbol.

If the formula has free variables, a **valuation** assigns each free variable an element of its domain.

If $D = \emptyset$, then:

$$\forall x \in D.p(x) \text{ is vacuously true,}$$

$$\exists x \in D.p(x) \text{ is false.}$$

Definition 2.2.4: Validity, Satisfiability, Unsatisfiability

Let E be a predicate logic formula.

- E is **valid** (a tautology) if it is true under every interpretation.
- E is **satisfiable** if it is true under at least one interpretation.
- E is **unsatisfiable** if it is false under every interpretation.

Example 2.2.2 (Valid and Satisfiable Formulas)

- $(\forall x \in D.a(x)) \text{ IMPLIES } \exists x \in D.a(x)$ is **valid**.
- $(\forall x \in D.a(x)) \text{ IMPLIES } a(c)$ is **valid**.
- $(\exists x \in D.a(x)) \text{ IMPLIES } \forall x \in D.a(x)$ is **satisfiable but not valid**.

Example interpretation: Let D be employees and $a(x)$ mean “ x is female”.

Then $(\exists x.a(x)) \text{ IMPLIES } \forall x.a(x)$ is false if not all employees are female, but true if $|D| = 1$.

Theorem 2.2.1 Quantifier Distribution Laws

$$\exists x(p(x) \text{ OR } q(x)) \equiv (\exists x p(x)) \text{ OR } (\exists x q(x))$$

$$\forall x(p(x) \text{ AND } q(x)) \equiv (\forall x p(x)) \text{ AND } (\forall x q(x))$$

If x does not occur in E , then:

$$\exists x(E \text{ AND } q(x)) \equiv E \text{ AND } \exists x q(x),$$

$$\forall x(E \text{ OR } q(x)) \equiv E \text{ OR } \forall x q(x).$$

Definition 2.2.5: Prenex Normal Form

A predicate logic formula is in **prenex normal form** if it has the structure

$$Q_1 x_1 \in D_1. Q_2 x_2 \in D_2. \cdots Q_k x_k \in D_k. E(x_1, \dots, x_k),$$

where:

- each Q_i is either $\forall$ or $\exists$,
- E contains no quantifiers.

Theorem 2.2.2 Negation of Quantifiers

$$\neg(\exists x.p(x)) \equiv \forall x.\neg p(x),$$

$$\neg(\forall x.p(x)) \equiv \exists x.\neg p(x).$$

Example 2.2.3 (Conversion to Prenex Normal Form)

Convert:

$$(f(x, c) \text{ OR } \exists y \in A. g(y)) \text{ IMPLIES } \forall z \in B. h(z).$$

Step 1:

$$(\exists y \in A. (f(x, c) \text{ OR } g(y))) \text{ IMPLIES } \forall z \in B. h(z).$$

Step 2:

$$\forall y \in A. ((f(x, c) \text{ OR } g(y)) \text{ IMPLIES } \forall z \in B. h(z)).$$

Step 3:

$$\forall y \in A. \forall z \in B. ((f(x, c) \text{ OR } g(y)) \text{ IMPLIES } h(z)).$$

This is prenex normal form.

2.3 Proof Techniques

Definition 2.3.1: Proposition, Axiom, Proof

A **proposition** is a statement that is either true or false.

An **axiom** is a proposition that we agree is true.

A **proof** is a convincing argument that a proposition is true. It consists of a sequence of axioms, previously proved propositions, and logical deductions.

A **logical deduction** applies an inference rule to derive a new proposition from axioms and previously proved propositions.

Definition 2.3.2: Substitution (Tautologies)

Let R be a tautology containing propositional variable P . If R' is obtained by replacing every occurrence of P in R by a formula Q , then R' is also a tautology.

Example:

$$R = (A \text{ OR } P) \text{ IMPLIES } (P \text{ OR } A), \quad Q = C \text{ AND } D.$$

Then

$$(A \text{ OR } (C \text{ AND } D)) \text{ IMPLIES } ((C \text{ AND } D) \text{ OR } A)$$

is a tautology.

Definition 2.3.3: Substitution of Equivalent Subformulas

Let S' be logically equivalent to S . If S is a subformula of R and R' is obtained by replacing some occurrences of S in R by S' , then R' is logically equivalent to R .

Definition 2.3.4: Modus Ponens

If P and $P \text{ IMPLIES } Q$ are true, then Q is true.

Formally:

$$P, \quad P \text{ IMPLIES } Q \quad \Rightarrow \quad Q.$$

Note: From $P \text{ IMPLIES } Q$ and Q , one cannot conclude P .

Definition 2.3.5: Specialization

If $c \in D$ and

$$\forall x \in D. a(x)$$

is true, then $a(c)$ is true.

Equivalently,

$$(\forall x \in D. a(x)) \text{ IMPLIES } a(c)$$

is a tautology.

Definition 2.3.6: Transitivity of Implication

If $P \text{ IMPLIES } Q$ and $Q \text{ IMPLIES } R$ are true, then $P \text{ IMPLIES } R$ is true.

Theorem 2.3.1 Direct Proof of Implication

To prove P IMPLIES Q :

1. Assume P .
2. Deduce Q .
3. Conclude P IMPLIES Q .

Theorem 2.3.2 Indirect Proof (Contrapositive)

Since

$$P \text{ IMPLIES } Q$$

is logically equivalent to

$$\text{NOT}Q \text{ IMPLIES } \text{NOT}P,$$

one may prove $\text{NOT}Q \text{ IMPLIES } \text{NOT}P$ to establish $P \text{ IMPLIES } Q$.

Theorem 2.3.3 Proof of a Disjunction

If P is proved, then

$$P \text{ OR } Q$$

follows.

Theorem 2.3.4 Proof of a Conjunction

If P and Q are both proved, then

$$P \text{ AND } Q$$

follows.

Theorem 2.3.5 Proof by Contradiction

To prove P :

1. Assume $\text{NOT}P$.
2. Derive a contradiction (a formula of the form $Q \text{ AND } \text{NOT}Q$).
3. Conclude P .

Lemma 2.3.1

If x is even, then x^2 is even.

Proof: Assume x is even. Then $x = 2k$ for some integer k . Hence

$$x^2 = (2k)^2 = 4k^2 = 2(2k^2),$$

which is even. ☺

Lemma 2.3.2

If x^2 is even, then x is even.

Proof: Suppose x is not even. Then x is odd, so $x = 2k + 1$ for some integer k . Then

$$x^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which is odd. Thus x^2 is not even. Therefore, if x^2 is even, then x is even. $\odot$

Theorem 2.3.6

The number $\sqrt{2}$ is irrational.

Proof: Assume, for contradiction, that $\sqrt{2}$ is rational. Then there exist relatively prime positive integers x and y such that

$$\sqrt{2} = \frac{x}{y}.$$

Then

$$x^2 = 2y^2.$$

Hence x^2 is even, so by the lemma, x is even. Thus $x = 2k$ for some integer k .

Substituting,

$$2y^2 = (2k)^2 = 4k^2,$$

so

$$y^2 = 2k^2.$$

Thus y^2 is even, and again by the lemma, y is even.

Therefore both x and y are divisible by 2, contradicting the assumption that they are relatively prime.

Hence $\sqrt{2}$ is irrational. $\odot$

2.4 Proof of Equivalences and Advanced Proof Techniques

Definition 2.4.1: Proof of Logical Equivalence

To prove

$$P \text{ IFF } Q,$$

it suffices to prove both

$$P \text{ IMPLIES } Q \quad \text{and} \quad Q \text{ IMPLIES } P.$$

More generally, to prove

$$P \text{ IFF } Q, \quad Q \text{ IFF } R, \quad P \text{ IFF } R,$$

it suffices to prove

$$P \text{ IMPLIES } Q, \quad Q \text{ IMPLIES } R, \quad R \text{ IMPLIES } P.$$

Proving

$$P \text{ IMPLIES } Q, \quad Q \text{ IMPLIES } R, \quad P \text{ IMPLIES } R$$

is **not** sufficient.

Definition 2.4.2: Proof by Cases

To prove

$$(P_1 \text{ OR } \dots \text{ OR } P_n) \text{ IMPLIES } Q,$$

it suffices to prove each

$$P_i \text{ IMPLIES } Q.$$

The cases $P_1, \dots, P_n$ may overlap.

Lemma 2.4.1

$$P \text{ IMPLIES } ((P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C))).$$

Proof: Assume P .

Since $C \text{ OR NOT}(C)$ is a tautology, consider two cases:

Case 1: Assume C . Then $P \text{ AND } C$, so $(P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C))$.

Case 2: Assume $\text{NOT}(C)$. Then $P \text{ AND NOT}(C)$, so $(P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C))$.

By proof by cases,

$$(P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C)).$$

Therefore

$$P \text{ IMPLIES } ((P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C))).$$



Lemma 2.4.2

$((P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C))) \text{ IMPLIES } P.$

Proof: Assume $(P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C)).$

Case 1: $P \text{ AND } C$ implies $P.$

Case 2: $P \text{ AND NOT}(C)$ implies $P.$

Thus by proof by cases, $P.$

**Definition 2.4.3: Generalization**

To prove

$$\forall x \in D.p(x),$$

let x be an arbitrary element of D and prove $p(x).$

Definition 2.4.4: Construction

To prove

$$\exists x \in D.p(x),$$

explicitly construct an element $x \in D$ and verify that $p(x)$ holds.

Definition 2.4.5: Instantiation

From

$$\exists x \in D.p(x),$$

introduce a new element $y \in D$ such that $p(y)$ holds.

Theorem 2.4.1

There is no largest integer.

Proof: Define

$$\text{largest}(y) = \text{NOT}(\exists x \in \mathbb{Z}.(x > y)).$$

Assume for contradiction that

$$\exists y \in \mathbb{Z}.\text{largest}(y).$$

Then there exists an integer y such that

$$\text{NOT}(\exists x \in \mathbb{Z}.(x > y)).$$

Let $x = y + 1$. Since $y + 1$ is an integer and $y + 1 > y$, we have

$$\exists x \in \mathbb{Z}.(x > y),$$

which contradicts

$$\text{NOT}(\exists x \in \mathbb{Z}.(x > y)).$$

Therefore,

$$\text{NOT}(\exists y \in \mathbb{Z}.\text{largest}(y)).$$



Lemma 2.4.3

P IMPLIES $((P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C)))$.

- 1 Assume P ; [assumption for implication proof](#)
- 2 C OR NOT(C); [tautology](#)
- 3 Assume C ; [assumption for implication proof](#)
- 4 P AND C ; [conjunction introduction 1,3](#)
- 5 $(P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C))$; [disjunction introduction 4](#)
- 6 C IMPLIES $((P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C)))$; [direct proof 3,5](#)
- 7 Assume NOT(C); [assumption for implication proof](#)
- 8 P AND NOT(C); [conjunction introduction 1,7](#)
- 9 $(P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C))$; [disjunction introduction 8](#)
- 10 NOT(C) IMPLIES $((P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C)))$; [direct proof 7,9](#)
- 11 $(P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C))$; [proof by cases 2,6,10](#)
- 12 P IMPLIES $((P \text{ AND } C) \text{ OR } (P \text{ AND NOT}(C)))$; [direct proof 1,11](#)

Lemma 2.4.4

$$\left\lfloor \frac{n+1}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil$$

Proof: Suppose n is even. Then $n = 2k$ for some integer k , so

$$\left\lfloor \frac{n+1}{2} \right\rfloor = \left\lfloor \frac{2k+1}{2} \right\rfloor = \left\lfloor k + \frac{1}{2} \right\rfloor = k = \lceil k \rceil = \left\lceil \frac{2k}{2} \right\rceil = \left\lceil \frac{n}{2} \right\rceil.$$

Thus, if n is even, then

$$\left\lfloor \frac{n+1}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil.$$

Suppose n is odd. Then $n = 2k + 1$ for some integer k , so

$$\left\lfloor \frac{n+1}{2} \right\rfloor = \left\lfloor \frac{2k+2}{2} \right\rfloor = \lfloor k+1 \rfloor = k+1 = \left\lceil k + \frac{1}{2} \right\rceil = \left\lceil \frac{2k+1}{2} \right\rceil = \left\lceil \frac{n}{2} \right\rceil.$$

Thus, if n is odd, then

$$\left\lfloor \frac{n+1}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil.$$

Since every integer is either even or odd, it follows that

$$\left\lfloor \frac{n+1}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil.$$

☺

Lemma 2.4.5

For all integers x , if x is even, then x^2 is even.

Proof: Let x be an integer.

Assume x is even. Then $x = 2k$ for some integer k , so

$$x^2 = (2k)^2 = 2(2k^2),$$

which is even.

Hence, x is even implies that x^2 is even. Since x is an arbitrary integer, for all integers x , if x is even, then x^2 is even. ☺

Definition 2.4.6: Largest

For any integer y , let

$$\text{largest}(y) = \text{NOT}(\exists x \in \mathbb{Z}.(x > y)).$$

Theorem 2.4.2

$\text{NOT}(\exists y \in \mathbb{Z}.\text{largest}(y)).$

- 1 Assume $\exists y \in \mathbb{Z}.\text{largest}(y)$; [assumption for contradiction](#)
- 2 Let y be an integer that satisfies $\text{largest}(y)$; [instantiation 1](#)
- 3 $\text{NOT}(\exists x \in \mathbb{Z}.(x > y))$; [definition 2](#)
- 4 Let $x = y + 1$; [construction](#)
- 5 $\forall z \in \mathbb{Z}.(z + 1 \in \mathbb{Z})$; [axiom](#)
- 6 $x = y + 1 \in \mathbb{Z}$; [specialization 5](#)
- 7 $\forall z \in \mathbb{Z}.(z + 1 > z)$; [axiom](#)
- 8 $x = y + 1 > y$; [specialization 7](#)
- 9 $\exists x \in \mathbb{Z}.(x > y)$; [construction 6,8](#)
- 10 $\text{NOT}(\exists y \in \mathbb{Z}.\text{largest}(y))$; [proof by contradiction 1,3,9](#)

2.5 Recursively defined sets

They have 1 or more base cases + constructor cases, that depend on previous cases.

Example 2.5.1

$\{0, 1\}^* = \text{set of all finite strings of bits}$

Base case: The empty string λ of length 0 is in $\{0, 1\}^*$.

Constructor case: if $s \in \{0, 1\}^*$, then s_0 and s_1 are in $\{0, 1\}^*$.

More generally if Σ is any finite set of letters, then $\Sigma^* = \text{set of finite strings of letters from } \Sigma$.

Example 2.5.2

$$\Sigma = \{[,]\}$$

$\text{Brkts} = \text{set of finite strings of matched brackets.}$

Base Case: The empty string $\lambda \in \text{Brkts}$

Constructor cases: If $s, t \in \text{Brkts}$, then $[s] \in \text{Brkts}$ and $st \in \text{Brkts}$.

Alternative constructor cases: If $s, t \in \text{Brkts}$, then $[s]t \in \text{Brkts}$

Example 2.5.3

$S = \text{set of syntatically correct formulas of propositional logic}$

Base case: propositional variables are in S .

Constructor cases: If $f, f' \in S$, then

$$\begin{aligned} NOT(f) &\in S \\ (f \text{ AND } f') &\in S \\ (f \text{ OR } f') &\in S \end{aligned}$$

Example 2.5.4

$M = \text{set of syntatically correct monotone formulas of propositional logic}$

Base case: propositional variables are in M

Constructor cases: if $f, f' \in M$, then

$$\begin{aligned} (f \text{ OR } f') &\in M \\ (f \text{ AND } f') &\in M \end{aligned}$$

M is the smallest set (this may not be said explicitly) of strings containing all the propositional variables that is closed under OR and AND

Example 2.5.5

Arithmetic expressions involving natural numbers, $+$ and $\times$.

Base case: $\mathbb{N} \subseteq A$

Constructor cases: if $e, f \in A$, then $e + f \in A$, $e \times f \in A$.

When a recursive definition allows an element to be constructed in more than 1 way. The definition is **ambiguous**. For example $3 \times 2 + 1$, it can be either $(3 \times (2 + 1)) \in A$ or $((3 \times 2) \in A + 1) \in A$

Structural Induction

Structural Induction is a proof technique used to prove that a property $P(x)$ holds for all elements x of a recursively defined set or data structure.

A structural induction proof consists of two parts:

1. **Base Case(s):** Show that $P(x)$ holds for each *base element* of the recursive definition.
2. **Inductive Step(s):** Assume that P holds for the immediate subcomponents of a recursively constructed element (this is the *induction hypothesis*), and use this assumption to prove that P holds for the element itself.

If both parts are proven, then $P(x)$ holds for all elements x in the recursively defined structure.

Example 2.5.6

Let S be recursively define set. Let $p : S \rightarrow \{T, F\}$ be a predicate. Prove $p(x)$ for all base cases of the definition of S . Prove $p(x)$ for the constructor cases of the definition of S assuming $p(y)$ is true for all components y of x .

$$\forall x \in S. p(x)$$

Proof: For all $f \in M$, let $\#_i(f)$ = "the number of occurrences of prop variables in f ". $\#_\lambda(f)$ = "the number of occurrence s of connectives in f ".

$$p(f) = "\#_v(f) = 1 + \#_c(f)"$$

Base Case: if f is a propositional variable, then $\#_v(f) = 1$ and $\#_c(f) = 0$, so $p(f)$.

Constructor cases: Let $f, f' \in M$ be arbitrary, assume $p(f)$ and $p(f')$, suppose $f'' = (f \text{ OR } f')$, then $\#_v(f'') = \#_v(f') + \#_v(f) = (1 + \#_c(f')) + (1 + \#_c(f))$ by induction hypothesis, and $= 1 + (\#_c(f) + \#_c(f') + 1)$. So $p(f'')$, and

$$\forall f \in M. p(f)$$

Smilarly, if $f'' = (f \text{ AND } f')$ then $p(f'')$



Weak induction as a special case of structural induction

Example 2.5.7

$\mathbb{N}$ can be defined recursively.

Base case: $0 \in \mathbb{N}$

Constructor case: if $n \in \mathbb{N}$, then $n + 1 \in \mathbb{N}$.

Base case: prove $p(0)$

Constructor case: let $n \in \mathbb{N}$ be arbitrary. Assume $p(n)$, prove $p(n + 1)$
then

$$\forall n \in \mathbb{N}.p(n)$$

Example 2.5.8

Recursive definition of $\mathbb{N} \times \mathbb{N}$ **Base case:** $(0, 0) \in \mathbb{N} \times \mathbb{N}$.

Constructor case: if $(m, n) \in \mathbb{N} \times \mathbb{N}$, then $(m + 1, n) \in \mathbb{N} \times \mathbb{N}$ and $(m, m + 1) \in \mathbb{N} \times \mathbb{N}$.

$$\forall m \in \mathbb{N}, \forall n \in \mathbb{N}.p(m, n)$$

Base case: prove $p(0, 0)$

Constructor case: let $(m, n) \in \mathbb{N} \times \mathbb{N}$ be arbitrary, assume $p(m, n)$, prove $p(m + 1, n)$ and $p(m, n + 1)$.

Now this is a weak version of induction. Considering the strong version:

Let $(m, n) \in \mathbb{N} \times \mathbb{N}$ be arbitrary, assume $\forall i \in \mathbb{N}, \forall j \in \mathbb{N}$,

$$[(i \leq m) \text{ AND } (j \leq n)] \text{ AND } [(i < m) \text{ OR } (j < n)] \text{ IMPLIES } p(i, j)$$

Prove $p(m, n)$, $\forall m \in \mathbb{N}, \forall n \in \mathbb{N}.p(m, n)$

Example 2.5.9

Define

$$Q(m) = \forall n \in \mathbb{N}.p(m, n)$$

To prove $\forall m \in \mathbb{N}(\forall n \in \mathbb{N}.p(m, n))$, it suffices to prove $\forall m \in \mathbb{N}.Q(m)$.

Let $m \in \mathbb{N}$ be arbitrary, assume $\forall i \in \mathbb{N}((i < m) \text{ IMPLIES } Q(i))$. Let $n \in \mathbb{N}$ be arbitrary, assume $\forall j \in \mathbb{N}, [(j < n) \text{ IMPLIES } p(m, j)]$

...

$p(m, n)$, so $\forall j \in \mathbb{N}.p(m, j)$ by strong induction. So $Q(m)$ by definition,

$$\forall m \in \mathbb{N}, Q(m) \text{ by strong induction}$$

Example 2.5.10 (Defining functions on recursively defined sets)

$M =$ set of syntactically correct monotone formulas of propositional logic
Recursive definition of $\#_v : M \rightarrow \mathbb{N}$

$\#_v(x) = 1$ for any propositional variable x

$$\#_v(f'') = \#_v(f') + \#_v(f)$$

if $f'' = (f' \text{ OR } f)$, or $f'' = (f' \text{ AND } f)$

$eval : A \rightarrow \mathbb{N}$, where $\forall f \in A, eval(f) =$ "the natural number represented by f "

Base case: $\forall f \in \mathbb{N}, eval(f) = f$

Constructor cases: $\forall e, f \in A$

$$eval(e + f) = eval(e) + eval(f)$$

$$eval(e \cdot f) = eval(e) \cdot eval(f)$$

However the problem here is that function $eval$ is not well defined, for example

$$\begin{aligned} eval(3 \times 2 + 1) &= eval(3) \times eval(2 + 1) \\ &= 3 \times eval(e) = 3 \times 3 = 9 \end{aligned}$$

while

$$\begin{aligned} eval(3 \times 2 + 1) &= eval(3 \times 2) + eval(1) \\ &= 6 + 1 = 7 \end{aligned}$$

Example 2.5.11 (Binary trees)

Let B denote the set of all binary trees.

Base case: The empty tree is in B .

Constructor case: if $t_1, t_2 \in B$ and r is a node, then it's in B . Let $left(t) = t_1$, $right(t) = t_2$, and $root(t) = r$.

$$\#_n : B \rightarrow \mathbb{N}$$

Base case: (empty tree) = 0

Constructor case: if $t \in B$ then

$$\#_n(t) = \#_n(left(t)) + \#_n(right(t)) + 1$$

Then $\#_n(t) =$ "the number of nodes in the tree"

Another example:

$$\#_l : B \rightarrow \mathbb{N}$$

Base case: (empty tree) = 0

Extra base case: $\#_l(\text{one node tree}) = 1$

Constructor case: $\#_l(t) = \#_l(left(t)) + \#_l(right(t))$, then $\#_l(t) =$ number of leaves in t

Theorem 2.5.1

A binary tree with n nodes has at most $\lceil n/2 \rceil$ leaves

Proof: Define $q : B \rightarrow \{T, f\}$ such that $\forall t \in B$

$$q(t) = \#_L(t) = \lceil \#_L(t)/2 \rceil$$

Let $t \in B$ be arbitrary

Base case: t is the empty tree, $\#_L(t) = 0$ and $\#_N(t) = 0$. $\lceil 0/2 \rceil = \lceil \#_N(t)/2 \rceil$

Base case 2: t is a one node tree

$$\#_L(t) = 1 \text{ and } \#_N(t) = 1$$

so $\lceil 1/2 \rceil = \lceil \#_N(t)/2 \rceil = 1$

Constructor case: t has at least 2 nodes. $\#_N(t) = 2$. Assume $q(\text{left}(t))$ and $q(\text{right}(t))$.

$$\#_L(\text{left}(t)) = \lceil \#_N(\text{left}(t))/2 \rceil$$

and

$$\#_L(\text{right}(t)) \leq \lceil \#_N(\text{right}(t))/2 \rceil$$

$$\begin{aligned} \#_L(t) &= \#_L(\text{left}(t)) + \#_L(\text{right}(t)) \\ &\leq \lceil \#_N(\text{left}(t))/2 \rceil + \lceil \#_N(\text{right}(t))/2 \rceil \\ &\leq \frac{\#_N(\text{left}(t)) + 1}{2} + \frac{\#_N(\text{right}(t)) + 1}{2} \\ &= \frac{\#_N(\text{left}(t)) + \#_N(\text{right}(t)) + 1 + 1}{2} \\ &= \frac{\#_N(t) + 1}{2} \end{aligned}$$

Since $\#_L(t) \in \mathbb{N}$, it follows that

$$\#_L(t) \leq \lfloor \frac{\#_N(t) + 1}{2} \rfloor = \lceil \frac{\#_N(t)}{2} \rceil$$

☺

Example 2.5.12

Let E_i be the set of elements in S because of the base cases. For $i \geq 1$, let E_i be the set of elements in S because of applying the constructor cases i times.

Let $q : \mathbb{N} \rightarrow \{T, F\}$ be $q(i) = "\forall i \in E_i.p(i)"$. Then $\forall i \in \mathbb{N} q(i)$ means the same as $\forall i \in S.p(i)$. Since $S = \bigcup \{E_i | i \in \mathbb{N}\}$

Theorem 2.5.2

Every integer greater than 1 can be written as a product of primes.

Proof: To obtain a contradiction, suppose the claim is false. Let n be the smallest integer in $M = \{i \in \mathbb{Z} | i > 1\}$ that cannot be written as a product of prime.

If n is prime, it is a product of one prime, itself. So n is composite. Thus $\exists k \in M$ and $\exists m \in M$ such that $k \cdot m = n$. Since $k, m > 1$, it follows that $m < n$.

Hence k and m can both be written as products of prime due to the minimality of n . Thus $n = m \cdot k$, can be written as product of primes. This contradicts the definition of n . Hence the claim is true. $\odot$

Chapter 3

Well Ordered Principles

Definition 3.0.1: Partially ordered set

A set S is **partially ordered** if there is a binary predicate $r : S \times S \rightarrow \{T, F\}$ such that $\forall x, y, z \in S$:

- $r(x, x) = T$, reflexive
- $(r(x, y) \text{ AND } r(y, x)) \text{ IMPLIES } (x = y)$, assymetric
- $(r(x, y) \text{ AND } r(y, z)) \text{ IMPLIES } r(x, z)$, transitivity

It is called a partial order

Definition 3.0.2: Totally ordered set

S is **totally ordered** if there is a partial order $r : S \times S \rightarrow \{T, F\}$ such that $\forall x, y \in S$ $r(x, y) \text{ OR } r(y, x)$ (comparability).

r is called a **total order**.

Definition 3.0.3: Well ordered set

A totally ordered set is **well ordered** if every nonempty subset of S' of S has a smallest element m . (i.e. $m \in S'$ and for all $a \in S'$, $r(m, a) = T$)

Any nonempty subset of a well ordered set is well ordered.

Example 3.0.1

For all $x, y \in \mathbb{Z}$, let $x \leq y$ IFF $(|x| < |y|) \text{ OR } [(|x| = |y|) \text{ AND } (x < y)]$. So $\mathbb{Z}$, $\leq$ is well ordered.

Example 3.0.2 (Well ordering proof)

For $\mathbb{Q}^+$, consider the order based on $\max\{\text{numerator}, \text{denominator}\}$ when written in reduced form. If $\leq$ is a well ordering, then $x < y$ means $(x \leq y) \text{ AND } (x \neq y)$.

Suppose S is a well ordered set, $\leq$.

To obtain a contradiction, suppose $\forall e \in S, p(e)$ is false.
Let $C = \{e \in S \mid P(e) = F\}$ the set of counterexamples.
 $C \neq \emptyset$ by definition of C , 1,2.
Let e be the smallest element of C , e exists since S is well ordered.
Let $e' = \dots, e' \in C, e' \in C, e' < e$.
This is a contradiction to the definition of e .
 $\forall e \in S. p(e)$ Proof by contradiction.

Theorem 3.0.1

$\sqrt{2}$ is not rational.

Proof: We will conduct a proof by the Well-Ordering Principle. To obtain a contradiction, suppose $\sqrt{2}$ is rational. Then there exist $p, q \in \mathbb{Z}^+$ such that $\sqrt{2} = p/q$.

Let

$$C = \{k \in \mathbb{Z}^+ \mid \exists m \in \mathbb{Z}^+, \sqrt{2} = m/k\}$$

Since $\mathbb{Z}^+$ is well-ordered and $C \subseteq \mathbb{Z}^+$ is nonempty, C must have a least element. Let q be the smallest element of C . By the definition of C , there exists $p \in \mathbb{Z}^+$ such that $\sqrt{2} = p/q$. Squaring both sides and rearranging gives

$$2q^2 = p^2$$

Since $2q^2$ is even, p^2 must be even, which implies p is even. Hence, $p = 2r$ for some $r \in \mathbb{Z}^+$. Substituting this into our equation, we get $2q^2 = (2r)^2 = 4r^2$, which simplifies to

$$q^2 = 2r^2$$

Similarly, since $2r^2$ is even, q^2 is even, and thus q is even. So $q = 2s$ for some $s \in \mathbb{Z}^+$. Since $\sqrt{2} = p/q = 2r/2s = r/s$, it follows by definition that $s \in C$. However, $s = q/2 < q$, which contradicts the fact that q is the smallest element of C . Therefore, $\sqrt{2}$ is irrational. ☹

3.1 Countable and Uncountable Sets

Definition 3.1.1: Surjective

A function $f : A \rightarrow B$ is **surjective** means :

$$\forall y \in B, \exists x \in A. (f(x) = y)$$

If A and B are finite sets, then $|B| \leq |A|$.

Definition 3.1.2: Countable

A set C is **countable** if it is empty or there is a surjective function from $\mathbb{N}$ to C .

Theorem 3.1.1

Every finite set is countable

Proof: Suppose C is a finite nonempty set. Arrange the elements of C in some order,

$$C_0, C_1, \dots, C_{n-1}$$

where $n = |C|$. Define $f : \mathbb{N} \rightarrow C$ as follows:

$$f(i) = \begin{cases} c_i & \text{if } i \leq n-1 \\ c_{n-1} & \text{if } i > n-1 \end{cases}$$

☺

Theorem 3.1.2

If A and B are countable, then $A \cup B$ is countable.

Proof: Since A and B are countable, there exist surjective functions $f : \mathbb{N} \rightarrow A$ and $g : \mathbb{N} \rightarrow B$. We define a function $h : \mathbb{N} \rightarrow A \cup B$ by cases:

$$h(n) = \begin{cases} f\left(\frac{n}{2}\right) & \text{if } n \text{ is even} \\ g\left(\frac{n-1}{2}\right) & \text{if } n \text{ is odd} \end{cases}$$

To show h is surjective, let $x \in A \cup B$.

- If $x \in A$, then $\exists i \in \mathbb{N}$ such that $f(i) = x$. Then $h(2i) = f(i) = x$.
- If $x \in B$, then $\exists i \in \mathbb{N}$ such that $g(i) = x$. Then $h(2i+1) = g(i) = x$.

In either case, x is in the image of h . Thus h is surjective, and $A \cup B$ is countable.

Sub-claim: If A is countable and $B \subseteq A$, then B is countable.

If B is finite, it is countable by definition. Suppose B is infinite. Since A is countable, we can list its elements as $a_1, a_2, a_3, \dots$. We define a sequence for B by letting b_1 be the first element in the list of A that is also in B . Let b_2 be the next such element, and so on. Specifically, define $n_1 = \min\{n \in \mathbb{N} : a_n \in B\}$ and $n_k = \min\{n > n_{k-1} : a_n \in B\}$. Then the map $k \mapsto a_{n_k}$ is a bijection from $\mathbb{N}$ to B . ☺

Theorem 3.1.3

If A and B are countable, then $A \times B \rightarrow \{(a, b) | (a \in A) \text{ AND } (b \in B)\}$ is countable.

Proof: Let

$$f(k) = (x, y)$$

if location (x, y) at this infinite array contains the number $k \in \mathbb{N}$

☺

Lemma 3.1.1

If A is nonempty and countable, there is a surjective function $f : A \rightarrow B$, then B is countable.

Proof: Since A is countable and nonempty, there is a surjective function $g : \mathbb{N} \rightarrow A$. Consider $h : \mathbb{N} \rightarrow B$, defined by

$$h(i) = f(g(i)) \quad \forall i \in \mathbb{N}$$

Let $b \in B$ be arbitrary, since f is surjective, $\exists a \in A. f(a) = b$. Since g is surjective, $\exists n \in \mathbb{N}. g(n) = a$. Hence $h(n) = f(g(n)) = f(a) = b$. Then $\forall b \in B, \exists a \in \mathbb{N}. h(a) = b$. Thus h is surjective. $\odot$

Example 3.1.1

Let $\{0, 1\}^* =$ the set of finite binary sequences. Let

$$g(i, j) = \begin{cases} j\text{'th lexicographically smallest binary string of length } i & 1 \leq j \leq 2^i \\ \lambda & \text{otherwise} \end{cases}$$

Theorem 3.1.4

A countable union of countable sets is countable.

Proof: C is a countable collection of sets. Then $\exists f : \mathbb{N} \rightarrow C$ that is surjective. Each $S \in C$ is countable, so $\exists g_s : \mathbb{N} \rightarrow S$ that is surjective. Consider the function $h : \mathbb{N} \times \mathbb{N} \rightarrow \{S | S \in C\}$, defined by $h(i, j) = g_{f(i)}(j)$, the j th element of the i th set in C . h is surjective. $\cup\{S | S \in C\} = \cup\{f(i) | i \in \mathbb{N}\}$ is countable. $\odot$

Theorem 3.1.5

Let $\{0, 1\}^w =$ set of all infinite binary sequences.

$$\{0, 1\}^w \text{ is uncountable}$$

Proof: Suppose $\{0, 1\}^w$ is countable, then there is a surjective function $b : \mathbb{N} \rightarrow \{0, 1\}^w$. Let $B : \mathbb{Z} \rightarrow \{0, 1\}^w$ be defined by $B(i) = b(i - 1)$. B is a function from $\mathbb{Z}^+$ to $\{0, 1\}^w$, let $D =$ the sequence of bits on the diagonal.

$$D(i) = (B(i))(i)$$

Let $C =$ the complement of D , $C(i) = 1 - D(i) = 1 - (B(i))(i)$ for all $i \in \mathbb{Z}^+$. Since $C \in \{0, 1\}^w$ and B is surjective, there exists $j \in \mathbb{Z}^+$ such that $B(j) = C$. Then $(B(j))(j) = C(j) = 1 - (B(j))(j)$. This is a contradiction. Thus $\{0, 1\}^w$ is uncountable. $\odot$

3.2 Merge Sort

Definition 3.2.1: Partial Correctness

An algorithm is **partially correct** if the following holds:
If the preconditions hold and the algorithm is executed, and if the algorithm eventually halts, then the postconditions hold.

Definition 3.2.2: Termination

An algorithm **terminates** if, whenever the preconditions hold and the algorithm is executed, the algorithm eventually halts.

Definition 3.2.3: Total Correctness

An algorithm is **totally correct** if it is both partially correct and terminating.
Equivalently,

$$\text{total correctness} = \text{partial correctness} + \text{termination}.$$

Definition 3.2.4: Specification for Merging Two Arrays

We specify an algorithm that merges two arrays A and B .

For example:

```
1  $i \leftarrow 1$ 
2  $j \leftarrow 1$ 
3 while  $(i \leq m)$  or  $(j \leq n)$  do
4   if  $A[i] \leq B[j]$ 
5     then  $C[k] \leftarrow A[i]$ 
6        $i \leftarrow i + 1$ 
7   else  $C[k] \leftarrow B[j]$ 
8        $j \leftarrow j + 1$ 
9    $k \leftarrow k + 1$ 
10 return  $C$ 
```

Note:-

Preconditions

- The elements of $A[1..m]$ and $B[1..n]$ come from a totally ordered domain.
- $A[1..m]$ and $B[1..n]$ are in nondecreasing order.
- $m \leq \text{length}(A)$.
- $n \leq \text{length}(B)$.
- $m, n \in \mathbb{N}$.

Note:-

Postconditions

- The algorithm returns an array C .
- The elements of $C[1..m+n]$ are in nondecreasing order.
- The multiset of elements in $C[1..m+n]$ is the union of the multisets of elements in $A[1..m]$ and $B[1..n]$.
- A , B , m , and n remain unchanged.

Definition 3.2.5: Loop Invariant

A **loop invariant** is a predicate that is true each time a particular place in loop is reached.

Loop Invariants

1. $i + j - 1 = k$
2. The elements in $C[1..k-1]$ are in nondecreasing order.
3. The multiset of elements in $C[1..k-1]$ equals the multiset of elements in $A[1..i-1]$ and $B[1..j-1]$.
4. $1 \leq i \leq m+1$ and $1 \leq j \leq n+1$.

Proof: We prove by induction on the number of iterations of the loop that have been performed that the loop invariants hold after each iteration.

For all $t \in \mathbb{N}$, let $P(t)$ be the statement:

“If the loop has been executed t times, then immediately after the t th iteration, the loop invariants are true.”

Here, “immediately after iteration 0” means immediately before iteration 1.

Base case: $t = 0$.

Before the first iteration, the initialization gives

$$i = j = k = 1.$$

Hence

$$i + j - 1 = 1 + 1 - 1 = 1 = k,$$

so invariant 1 holds, and clearly

$$1 \leq i \leq m+1, \quad 1 \leq j \leq n+1,$$

so invariant 4 holds.

Also, $C[1..k-1] = C[1..0]$, $A[1..i-1] = A[1..0]$, and $B[1..j-1] = B[1..0]$ are all empty arrays. Therefore $C[1..k-1]$ is trivially in nondecreasing order, and its multiset equals the union of the multisets of $A[1..i-1]$ and $B[1..j-1]$. Thus invariants 2 and 3 also hold.

So $P(0)$ is true.

Induction step: Let $t \geq 1$ be arbitrary, and assume $P(t-1)$ is true. We must prove $P(t)$.

Let i', j', k' denote the values of i, j, k at the beginning of the t th iteration, and let i'', j'', k'' denote their values at the end of this iteration.

Since the t th iteration is actually executed, the loop condition is true at the start of the iteration. Thus the relevant branch of the algorithm is well-defined.

Case 1: The algorithm copies $A[i']$ into $C[k']$.

Then

$$C[k'] \leftarrow A[i'], \quad i'' = i' + 1, \quad j'' = j', \quad k'' = k' + 1.$$

- **Invariant 1:** By the induction hypothesis,

$$i' + j' - 1 = k'.$$

Hence

$$i'' + j'' - 1 = (i' + 1) + j' - 1 = i' + j' = k' + 1 = k''.$$

- **Invariant 2:** By the induction hypothesis, $C[1..k' - 1]$ is in nondecreasing order. We append $A[i']$ as the new last element.

If $k' = 1$, then $C[1..k'' - 1] = C[1..1]$ has only one element, so it is automatically in nondecreasing order.

If $k' > 1$, let x be the last element of $C[1..k' - 1]$. By invariant 3, every element already placed in $C[1..k' - 1]$ comes from $A[1..i' - 1]$ or $B[1..j' - 1]$.

Because $A[1..m]$ is in nondecreasing order, every element of $A[1..i' - 1]$ is $\leq A[i']$.

Also, if some elements of B have already been copied, then they lie in $B[1..j' - 1]$. Since the algorithm chooses $A[i']$ in this case, we have $A[i'] \leq B[j']$, and because $B[1..n]$ is in nondecreasing order, every element of $B[1..j' - 1]$ is $\leq B[j']$. The last copied element x is the largest element of $C[1..k' - 1]$, so in particular $x \leq A[i']$.

Therefore appending $A[i']$ preserves nondecreasing order, so $C[1..k'' - 1]$ is in nondecreasing order.

- **Invariant 3:** By the induction hypothesis, the multiset of elements in $C[1..k' - 1]$ equals the union of the multisets of elements in $A[1..i' - 1]$ and $B[1..j' - 1]$.

After this iteration, we append exactly the element $A[i']$ to $C[1..k' - 1]$, and i increases from i' to $i'' = i' + 1$, while $j'' = j'$. Thus the multiset of elements in $C[1..k'' - 1]$ equals the union of the multisets of elements in

$$A[1..i'' - 1] = A[1..i'] \quad \text{and} \quad B[1..j'' - 1] = B[1..j' - 1].$$

- **Invariant 4:** By the induction hypothesis,

$$1 \leq i' \leq m + 1, \quad 1 \leq j' \leq n + 1.$$

Since this branch copies $A[i']$, we must have $i' \leq m$, so

$$1 \leq i'' = i' + 1 \leq m + 1.$$

Also,

$$1 \leq j'' = j' \leq n + 1.$$

So invariant 4 holds.

Case 2: The algorithm copies $B[j']$ into $C[k']$.

Then

$$C[k'] \leftarrow B[j'], \quad j'' = j' + 1, \quad i'' = i', \quad k'' = k' + 1.$$

By the same argument as in Case 1, with the roles of A and B and of i and j interchanged, invariants 1–4 all hold after this iteration.

Therefore $P(t)$ is true.

By mathematical induction, $P(t)$ is true for all $t \in \mathbb{N}$. Hence the loop invariants hold throughout the execution of the algorithm. $\odot$

Proof of partial correctness

Suppose the preconditions are satisfied and the $MERGE(A, m, B, n)$ algorithm is run and it terminates. From the termination condition of the while loop, $(i > m)$ and $(j > n)$. Immediately after the last iteration, the loop invariants hold. In particular, $i \leq m + 1$ and $j \leq n + 1$. Hence $i = m + 1$ and $j = n + 1$. $h = i + j - 1 = (m + 1) + (n + 1) - 1 = m + n + 1$.

The elements of $C[1..(k - 1)]$ = the elements of $C[1..m + n]$ are in nondecreasing order + the multiset of values in term is the union of the multiset of values in $A[1..(i - 1)] \cup B[1..(j - 1)] = A[1..m] \cup B[1..n]$ by the loop invariants.

Termination

Every iteration either i is incremented or j is incremented. By the loop invariants, $i \leq m + 1$ and $j \leq n + 1$, so eventually $i > m$ and $j > n$. This is the termination condition for the loop.

3.2.1 Example 2: Multiplication

```

1  w ← m
2  y ← n
3  z ← 0
4  while w ≠ 0 do
5      if w (mod 2) = 1 then z ← z + y
6      w ← w div 2
7      y ← y · 2
8  return z
```

Precondition

$$m, n \in \mathbb{N}$$

Postcondition

$$z = m \cdot n$$

Loop Invariants

Let w_i, y_i, z_i be the values of w, y, z immediately after the i th iteration of the loop.

$$y_i = n2^i$$

$$w_i = \lfloor m/2^i \rfloor$$

z_i = the sum of the first i rows between the two lines = $n \times$ (the i least significant bits of m)

$$b_j = \begin{cases} 1 & \text{if the } j\text{th least significant bit of } m \text{ is } 1 \\ 0 & \text{if the } j\text{th least significant bit of } m \text{ is } 0 \end{cases}$$

Example 3.2.1 (Multiplication Example)

Compute 12×5 using the algorithm above.

We have $m = 12 = (1100)_2$ and $n = 5$, so $b_0 = 0, b_1 = 0, b_2 = 1, b_3 = 1$.

Iteration	w	y	z
0	$(1100)_2$	5	0
1	$(110)_2$	10	0
2	$(11)_2$	20	20
3	$(1)_2$	40	20
4	$(0)_2$	80	60

The algorithm returns $z = 60 = (111100)_2 = 12 \times 5$. ✓

Lemma 3.2.1

If $m, n \in \mathbb{N}$, then $z = n \times m - y \times w$ is a loop invariant.

Proof: Let $P(i)$ = "immediately after the i th iteration of the loop, $z = n \times m - y \times w$ ". ☺

Partial correctness: If $m, n \in \mathbb{N}$ and $MULT(m, n)$ is run till halts, then when it halts, $z = n \times m$.

Proof: When the loop terminates, $w = 0$. By the lemma, $z = n \times m - y \times w = n \times m$. ☺

Lemma 3.2.2

If $n, m \in \mathbb{N}$ and $MULT(m, n)$ is run, it eventually terminates.

Proof: Suppose the loop doesn't terminate. Consider the infinite sequence $w_0, w_1, w_2, \dots \in \mathbb{N}$, none of the w_i s are 0 since the loop doesn't terminate.

$$w_{i+1} = w_i \text{ div } 2 < w_i$$

thus this is an infinite decreasing sequence of negative integer. This contradicts the fact that $\mathbb{N}$ is well-ordered. Hence, the loop terminates. $\odot$

Full proof of Correctness

We prove that the algorithm correctly computes the product $m \cdot n$ using the loop invariants established above.

- **Base Case** ($k = 0$): Before the loop starts ($k = 0$):
 - $w_0 = m = \lfloor m/2^0 \rfloor$ (Invariant holds)
 - $y_0 = n = n \cdot 2^0$ (Invariant holds)
 - $z_0 = 0$. Since $m \pmod{2^0} = 0$, we have $n \cdot 0 = 0$. (Invariant holds)
- **Inductive Step:** Assume the invariants hold for iteration k . We show they hold for iteration $k + 1$. The loop body executes the following steps:
 1. **Update z :** If $w_k \pmod{2} = 1$, then $z_{k+1} = z_k + y_k$. Otherwise, $z_{k+1} = z_k$. Note that $w_k \pmod{2} = \lfloor m/2^k \rfloor \pmod{2}$ is exactly the k -th bit of m , denoted b_k . Thus, we can write:

$$z_{k+1} = z_k + b_k \cdot y_k$$

Substituting the inductive hypotheses:

$$z_{k+1} = n \cdot (m \pmod{2^k}) + b_k \cdot (n \cdot 2^k) = n [(m \pmod{2^k}) + b_k \cdot 2^k]$$

The term in the brackets represents the value formed by the first $k + 1$ bits of m , which is $m \pmod{2^{k+1}}$. Therefore, $z_{k+1} = n \cdot (m \pmod{2^{k+1}})$.

2. **Update w :** $w_{k+1} = \lfloor w_k/2 \rfloor = \lfloor \lfloor m/2^k \rfloor / 2 \rfloor = \lfloor m/2^{k+1} \rfloor$.
3. **Update y :** $y_{k+1} = y_k \cdot 2 = (n \cdot 2^k) \cdot 2 = n \cdot 2^{k+1}$.

All invariants are maintained.

- **Termination:** The loop terminates when $w_k = 0$. From the invariant, this implies $\lfloor m/2^k \rfloor = 0$, meaning $2^k > m$. When $2^k > m$, the value represented by the "first k least significant bits of m " is simply m itself ($m \pmod{2^k} = m$). Substituting this into the invariant for z at termination:

$$z_k = n \cdot (m \pmod{2^k}) = n \cdot m$$

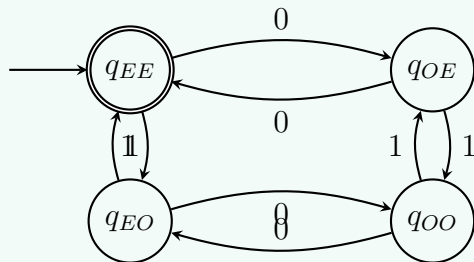
Thus, the algorithm returns the correct product.

3.3 Finite Automata

A deterministic finite state automata (DFA or DFSA) has a finite set of state Q , an initial state $q_0 \in Q$, a set of finite states F , each directed edge represents a transition from a state to another state. The label on the edge says that letter causes the transition.

Example 3.3.1 (DFA Illustration: Even Number of 0s and 1s)

$$L = \{w \in \{0,1\}^* \mid \#_0(w) \text{ is even and } \#_1(w) \text{ is even}\}$$



Here each state records parities of counts of 0s and 1s:

$$q_{EE} = (\text{even 0s, even 1s}),$$

$$q_{OE} = (\text{odd 0s, even 1s}),$$

$$q_{EO} = (\text{even 0s, odd 1s}),$$

$$q_{OO} = (\text{odd 0s, odd 1s}).$$

The start state is q_{EE} , and the only accepting state is q_{EE} .

Given an input string $x \in \Sigma^*$, the finite automaton starts in the initial state, reads the input string 1 letter at a time (from left to right) it changes state according to the transition function (following the edge labelled by the letter). When all the letters in x have been read, the DFA accepts x if it is in a final state, otherwise it rejects x .

An informal definition:

$$L(M) = \{x \in \Sigma^* \mid M \text{ accepts } x\}$$

recursively define the extended transition function

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

Base case: $\delta^*(q, \lambda) = q$. **Constructor case:** for all $x \in \Sigma^*$ and $a \in \Sigma$, $\delta^*(q, xa) = \delta(\delta^*(q, x), a)$.

One can prove by induction that for all $x, y \in \Sigma^*$ and all $q \in Q$,

$$\delta^*(q, xy) = \delta^*(\delta^*(q, x), y)$$

Formalizing the definition:

$$L(M) = \{x \in \Sigma^* \mid \delta^*(q_0, x) \in F\}$$

For example

$$L(M) = \{x \in \{0,1\}^* \mid x \text{ begins with 1 or begins and ends with 0}\}$$

Prove by structural induction for all $x \in \Sigma^*$,

$$P(x) = "\forall i \in \{0, 1, 2, 3\} \mid (x \in L_i) \text{ IFF } \delta^*(q_0, x) = q_i"$$

or by induction for all $n \in \mathbb{N}$

$$P'(n) = "\forall x \in \Sigma^n. P(x)"$$

$$L_0 = \{\lambda\}$$

$$L_1 = \{x \in \Sigma^* \mid x \text{ begins with } 1\}$$

$$L_2 = \{x \in \Sigma^* \mid x \text{ begins with } 1 \text{ ends with } 0\}$$

$$L_3 = \{x \in \Sigma^* \mid x \text{ begins with } 0 \text{ ends with } 1\}$$

Definition 3.3.1: Nondeterministic Finite Automaton (NFA)

An NFA is a 5-tuple

$$M = (Q, \Sigma, s, \delta, F)$$

where:

- Q is a set of states
- Σ is the alphabet
- $s \in Q$ is the start state
- $F \subseteq Q$ is the set of accepting states
- $\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q)$

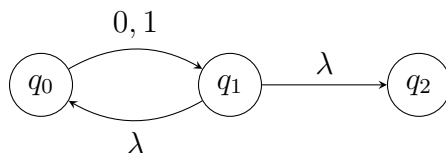
Note:-

$\delta(q, a)$ is the set of states that the machine can go to from state q when reading $a \in \Sigma \cup \{\lambda\}$.

Definition 3.3.2: Extended Transition Function δ^*

$\delta^*(q, x)$ = set of all possible states the machine may be in if it starts in q and reads x

$\delta^*(q, x) = \{q' \in Q \mid \text{there exists a path from } q \text{ to } q' \text{ such that the concatenation of labels is } x\}$



$$\delta(q_0, 1) = \{q_0, q_1\}$$

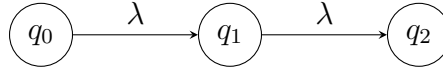
Definition 3.3.3: λ -closure

For all $q \in Q$, let

$E(q)$ = set of states reachable from q by following any number of λ -transitions

Note:-

At most $|Q| - 1$ λ -transitions are needed (no cycles add new states).



$$E(q_0) = \{q_0, q_1, q_2\}$$

3.3.1 Recursive Definition of δ^*

Proposition 3.3.1

$$\delta^*(q, \lambda) = E(q)$$

Proposition 3.3.2

For all $a \in \Sigma$ and $x \in \Sigma^*$,

$$\delta^*(q, xa) = \bigcup \{E(q'') \mid \exists q' \in \delta^*(q, x) \text{ such that } q'' \in \delta(q', a)\}$$

Definition 3.3.4: Extension of δ^* to subsets

$$\delta^* : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$$

$$\delta^*(Q', x) = \bigcup_{q \in Q'} \delta^*(q, x)$$

Note:-

This allows us to track all possible states simultaneously.

3.4 NFA and DFA Have the Same Power

Theorem 3.4.1 Subset Construction

For every NFA

$$M = (Q, \Sigma, s, \delta, F),$$

there exists a DFA

$$M' = (Q', \Sigma, s', \delta', F')$$

such that

$$L(M) = L(M').$$

Idea of the proof. An NFA may be in many possible states after reading a string, while a DFA must be in exactly one state. So the main idea is:

make one DFA state represent a whole set of NFA states.

That is the key construction. If after reading a word x the NFA could be in any state of some set $S \subseteq Q$, then the DFA will simply use the single state S to remember all of those possibilities at once.

This is the important step because a DFA cannot branch, but it *can* store all branches together as one subset of states.

Proof: Let

$$M = (Q, \Sigma, s, \delta, F)$$

be an NFA. We will construct a DFA

$$M' = (Q', \Sigma, s', \delta', F')$$

and then prove that it accepts exactly the same language.

Step 1: Define the states of the DFA.

Since the DFA must remember *all possible current NFA states*, define

$$Q' = \mathcal{P}(Q),$$

the power set of Q .

Motivation. If the NFA could currently be in states q_1, q_3, q_5 , then the DFA should have one state representing exactly that information, namely the subset $\{q_1, q_3, q_5\}$.

Step 2: Define the start state.

If the NFA has no λ -moves, define

$$s' = \{s\}.$$

If the NFA allows λ -moves, let $E(s)$ be the λ -closure of s , and define

$$s' = E(s).$$

Motivation. Before reading any input, the NFA may already move along λ -transitions, so the DFA must begin by remembering every state reachable immediately in that way.

Step 3: Define the transition function.

For each subset $S \subseteq Q$ and each $a \in \Sigma$, define

$$\delta'(S, a) = \bigcup_{q \in S} \delta(q, a)$$

in the case with no λ -moves.

If λ -moves are allowed, define

$$\delta'(S, a) = E\left(\bigcup_{q \in S} \delta(q, a)\right),$$

where $E(\cdot)$ means take λ -closure of the whole set.

Motivation. Suppose the NFA could currently be in any state in S . After reading the symbol a , it could move from each $q \in S$ to any state in $\delta(q, a)$. So to record *all* possible next states, we must take the union over all $q \in S$.

This union is the heart of the construction: it compresses all nondeterministic branches into one deterministic update rule.

Step 4: Define the accepting states.

Let

$$F' = \{S \subseteq Q : S \cap F \neq \emptyset\}.$$

Motivation. The NFA accepts if *at least one* computation ends in an accepting state. Since the DFA state S represents all possible current NFA states, it should be accepting exactly when one of those states is accepting.

Now we prove that

$$L(M) = L(M').$$

We first prove the following claim.

Claim. For every string $x \in \Sigma^*$,

$$\delta'^*(s', x) = \delta^*(s, x),$$

where the left-hand side is the extended transition function of the DFA and the right-hand side is the set of all NFA states reachable after reading x (including λ -closures if applicable).

Proof of Claim. We use induction on the length of x .

Base case: $x = \lambda$.

For the DFA,

$$\delta'^*(s', \lambda) = s'.$$

By definition of s' , this equals $\{s\}$ in the no- λ case, or $E(s)$ in the λ -move case. That is exactly the set of NFA states reachable from s without reading any input. Hence

$$\delta'^*(s', \lambda) = \delta^*(s, \lambda).$$

Inductive step: Assume the claim holds for some string $x \in \Sigma^*$. Let $a \in \Sigma$. We prove it for xa .

By definition of extended transition for a DFA,

$$\delta'^*(s', xa) = \delta'(\delta'^*(s', x), a).$$

Using the induction hypothesis,

$$\delta'^*(s', xa) = \delta'(\delta^*(s, x), a).$$

Now let

$$S = \delta^*(s, x).$$

Then S is precisely the set of all NFA states that could be reached after reading x .

So after reading one more symbol a , the set of all possible new NFA states is:

$$\bigcup_{q \in S} \delta(q, a)$$

in the no- λ case, and

$$E\left(\bigcup_{q \in S} \delta(q, a)\right)$$

in the λ -move case.

But this is exactly how $\delta'(S, a)$ was defined. Therefore

$$\delta'^*(s', xa) = \delta^*(s, xa).$$

This completes the induction and proves the claim. $\triangle$

Now we finish the proof of the theorem.

Take any string $x \in \Sigma^*$. Then

$$x \in L(M') \iff \delta'^*(s', x) \in F'.$$

By definition of F' ,

$$\delta'^*(s', x) \in F' \iff \delta'^*(s', x) \cap F \neq \emptyset.$$

By the claim,

$$\delta'^*(s', x) \cap F \neq \emptyset \iff \delta^*(s, x) \cap F \neq \emptyset.$$

But this last condition means exactly that the NFA M accepts x . Hence

$$x \in L(M') \iff x \in L(M).$$

Therefore

$$L(M') = L(M).$$

So for every NFA there exists a DFA accepting the same language. $\odot$

Corollary 3.4.1

NFAs and DFAs recognize exactly the same class of languages, namely the regular languages.

Note:-

The DFA obtained from this construction may have as many as $2^{|Q|}$ states, because its states are subsets of the $|Q|$ NFA states.

Chapter 4

Regular Languages and the Pumping Lemma

4.1 Regular Languages

Definition 4.1.1: Regular Language

A language L is **regular** if it is the language accepted by some finite automaton, i.e. there exists a DFA or NFA M such that $\mathcal{L}(M) = L$.

4.2 Non-Regular Languages

Example 4.2.1 ($S = \{a^i b^i \mid i \geq 1\}$)

The language S is not regular.

Proof: (By contradiction.)

Suppose there exists a DFA

$$M = (Q, \{a, b\}, \delta, q_0, F) \quad \text{s.t.} \quad \mathcal{L}(M) = S.$$

Let $n = |Q|$. Consider the states $q_i = \delta^*(q_0, a^i)$ for $i = 0, 1, \dots, n$.

By the pigeonhole principle, $\exists 0 \leq i < j \leq n$ such that $q_i = q_j$.

Then:

$$\begin{aligned} \delta^*(q_0, a^i b^j) &= \delta^*(\delta^*(q_0, a^i), b^j) \\ &= \delta^*(q_i, b^j) \\ &= \delta^*(q_j, b^j) \in F \end{aligned}$$

Since $a^j b^j \in S = \mathcal{L}(M)$, this implies $a^i b^j \in \mathcal{L}(M)$, which contradicts the fact that $a^i b^j \notin S$ (since $i \neq j$). ⊖

4.3 The Pumping Lemma

Lemma 4.3.1 Pumping Lemma

For every regular language S , there exists $n \in \mathbb{Z}^+$ such that for all $x \in S$ with $|x| \geq n$, there exist $u \in \Sigma^*$, $v \in \Sigma^*$, $w \in \Sigma^*$ satisfying:

1. $v \neq \lambda$,
2. $|uv| \leq n$,
3. $x = uvw$, and
4. $\forall k \in \mathbb{N}, uv^k w \in S$.

Proof: Suppose S is regular. Then $S = \mathcal{L}(M)$ for some DFA $M = (Q, \Sigma, \delta, q_0, F)$.

Let $n = |Q|$. Let $x \in S$ where $|x| = m \geq n$. Let $x[1, \dots, i]$ denote the length- i prefix of x .

Consider the states

$$q_i = \delta^*(q_0, x[1, \dots, i]) \quad \text{for } 0 \leq i \leq n.$$

By the pigeonhole principle, $\exists 0 \leq i < j \leq n$ such that $q_i = q_j$.

Let:

$$\begin{aligned} u &= x[1, \dots, i] = x(1), \dots, x(i), \\ v &= x[i+1, \dots, j] = x(i+1), \dots, x(j), \\ w &= x[j+1, \dots, m] = x(j+1), \dots, x(m). \end{aligned}$$

Then $x = uvw$, $v \neq \lambda$ (since $i < j$), and $|uv| \leq n$ (since $j \leq n$).

Also, $\forall k \in \mathbb{N}, \delta^*(q_i, v^k) = q_i = q_j$.

Hence:

$$\begin{aligned} \delta^*(q_0, uv^k w) &= \delta^*(q_0, uvw) \\ &= \delta^*(q_0, x) \in F. \end{aligned}$$

Since $x \in S = \mathcal{L}(M)$, we have $uv^k w \in \mathcal{L}(M) = S$. ☺

4.4 Application: $T = \{a^m b^n \mid m \neq n\}$ is Not Regular

Theorem 4.4.1

$T = \{a^m b^n \mid m \neq n\}$ is not regular.

Proof: Suppose T is regular. Then by the Pumping Lemma, $\exists n \in \mathbb{N}$ such that $\forall x \in T$ with $|x| \geq n$, there exist $u \in \{0, 1\}^*$, $v \in \{0, 1\}^*$, $w \in \{0, 1\}^*$ satisfying:

$$(v \neq \lambda) \quad \text{and} \quad (|uv| \leq n) \quad \text{and} \quad (x = uvw) \quad \text{and} \quad (\forall k \in \mathbb{N}, uv^k w \in T).$$

Consider $x = a^{n!} b^{(n+1)!}$ where $n \geq 1$ (note $1! = 0! = 1$).

We have $x \in T$ since $n! \neq (n+1)!$ for all $n \geq 1$.

Then $\exists u, v, w \in \{a, b\}^*$ such that:

$$(v \neq \lambda) \quad \text{and} \quad (|uv| \leq n) \quad \text{and} \quad (x = uvw) \quad \text{and} \quad (\forall k \in \mathbb{N}, uv^k w \in T).$$

Note that $u = a^i$ and $v = a^j$ for some $i \in \mathbb{N}$ and $j \in \mathbb{Z}^+$ such that $i + j \leq n$.

Then $w = a^{n!-i-j} b^{(n+1)!}$.

Let $k = 1 + \frac{n(n!)}{j} \in \mathbb{Z}^+$ (this is an integer since $j \leq n$ divides $n!$).

But $uv^k w \notin T$ because:

$$uv^k w = a^m b^{(n+1)!}$$

where

$$\begin{aligned} m &= n! + (k-1)j \\ &= i + jk + n! - i - j \\ &= n! + \frac{n(n!)}{j} \cdot j \\ &= n! + n(n!) \\ &= n!(1+n) \\ &= (n+1)! \end{aligned}$$

Then $uv^k w = a^{(n+1)!} b^{(n+1)!} \notin T$.

This is a contradiction. So T is not regular.



Chapter 5

Closure Properties of Regular Languages

5.1 Closure Results

Theorem 5.1.1 Closure of Regular Languages

Suppose $L_1, L_2 \subseteq \Sigma^*$ are regular. Then:

1. $\Sigma^* - L_1 = \{x \in \Sigma^* \mid x \notin L_1\}$ is regular (complement),
2. $L_1 \cup L_2 = \{x \in \Sigma^* \mid (x \in L_1) \text{ or } (x \in L_2)\}$ is regular (union),
3. $L_1 \cap L_2 = \{x \in \Sigma^* \mid (x \in L_1) \text{ and } (x \in L_2)\}$ is regular (intersection),
4. $L_1 - L_2 = \{x \in \Sigma^* \mid (x \in L_1) \text{ and } (x \notin L_2)\}$ is regular (set difference).

That is, the family of regular languages is closed under these four operations.

5.1.1 Proof of Closure under Complement

Proof: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be a DFA such that $L_1 = \mathcal{L}(M_1)$.

Let $M' = (Q_1, \Sigma, \delta_1, q_1, Q_1 - F_1)$.

Claim: $\mathcal{L}(M') = \Sigma^* - L_1$.

Let $x \in \Sigma^*$. Then:

$$\begin{aligned} x \in \mathcal{L}(M) &\iff \delta^*(q_1, x) \in Q_1 - F_1 \\ &\iff \delta^*(q_1, x) \notin F_1 \\ &\iff x \notin \mathcal{L}(M_1) = L_1. \end{aligned}$$



5.1.2 Proof of Closure under Union

Proof: Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be DFAs such that $L_1 = \mathcal{L}(M_1)$ and $L_2 = \mathcal{L}(M_2)$.

WLOG, assume $Q_1 \cap Q_2 = \emptyset$ (since we can rename states).

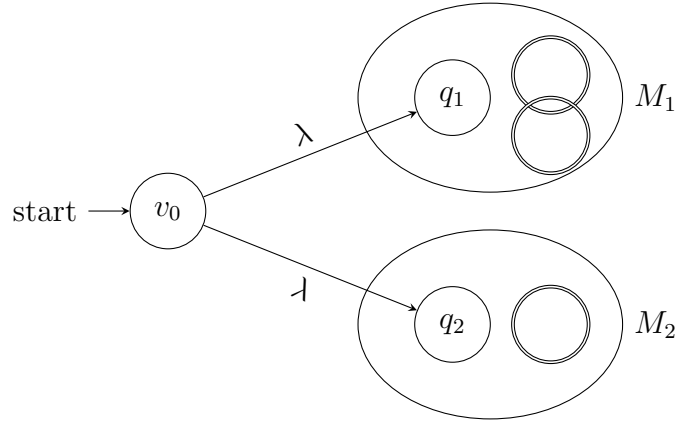
More formally, let

$$M = (Q_1 \cup Q_2 \cup \{v_0\}, \Sigma, \delta, v_0, F_1 \cup F_2)$$

where $v_0 \notin Q_1 \cup Q_2$ and the transition function δ is defined by:

$$\begin{aligned} \delta(v_0, \lambda) &= \{q_1, q_2\}, \\ \delta(v_0, a) &= \emptyset \quad \forall a \in \Sigma, \\ \delta(q, \lambda) &= \emptyset \quad \forall q \in Q_1 \cup Q_2, \\ \delta(q, a) &= \delta_1(q, a) \quad \forall q \in Q_1, a \in \Sigma, \\ \delta(q, a) &= \delta_2(q, a) \quad \forall q \in Q_2, a \in \Sigma. \end{aligned}$$

This is an NFA M whose construction looks like:



Claim: $\mathcal{L}(M) = L_1 \cup L_2$.

Let $x \in L_1$. Then $x \in \mathcal{L}(M_1)$, and the only path from q_1 labelled by x leads to a state in F_1 . So the path from v_0 to q_1 (labelled by λ) followed by π is a path from v_0 to $F_1 \subseteq F$. Hence $x \in \mathcal{L}(M)$.

Similarly, if $x \in L_2$, then $x \in \mathcal{L}(M)$. Hence $L_1 \cup L_2 \subseteq \mathcal{L}(M)$.

Now let $x \in \mathcal{L}(M)$. Then there exists a path labelled by x from v_0 to $F_1 \cup F_2$.

Case 1: There is a path π from v_0 to $f_1 \in F_1$ labelled by x . It starts with a λ -transition from v_0 to q_1 or q_2 . Since there are no transitions from Q_1 to Q_2 or Q_2 to Q_1 , π starts with a λ -transition to q_1 . Hence $x \in \mathcal{L}(M_1) = L_1$.

Similarly, if there is a path from v_0 to $f_2 \in F_2$ labelled by x , then $x \in \mathcal{L}(M_2) = L_2$.

So $\mathcal{L}(M) \subseteq L_1 \cup L_2$. ☺

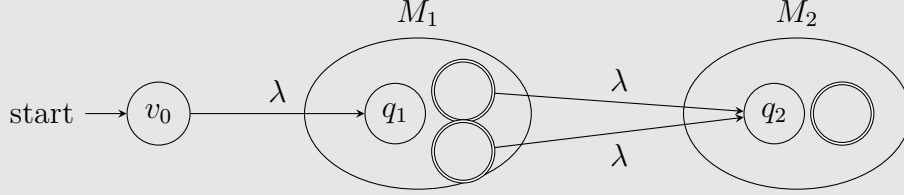
5.1.3 Closure under Intersection

Note:-

The construction for concatenation $L_1 \cdot L_2$ would give:

$$\mathcal{L}(M') = \{x \mid \exists x' \in L_1, \exists x'' \in L_2 \text{ s.t. } x = x'x''\} = L_1 \cdot L_2 \neq L_1 \cap L_2.$$

The wrong construction looks like this:



The correct approach for intersection uses De Morgan's law:

$$L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2)).$$

Since regular languages are closed under complement and union, $L_1 \cap L_2$ is regular.

Alternative: Product Construction

Note:-

Alternatively, let $M = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2)$ where $\delta : (Q_1 \times Q_2) \times \Sigma \rightarrow Q_1 \times Q_2$ is defined by:

$$\delta((q, q'), a) = (\delta_1(q, a), \delta_2(q', a)) \quad \forall q \in Q_1, q' \in Q_2, a \in \Sigma.$$

5.1.4 Closure under Set Difference

$$L_1 - L_2 = L_1 \cap (\Sigma^* - L_2).$$

Since regular languages are closed under $\cap$ and complement, $L_1 - L_2$ is regular.

5.2 Closure under Concatenation and Kleene Star

Define:

$$\begin{aligned} L_1^1 &= L_1, \\ L_1^{i+1} &= L_1^i \cdot L_1 = L_1 \cdot L_1^i, \\ L_1^+ &= \bigcup \{L_1^i \mid i \in \mathbb{Z}^+\}, \\ L_1^* &= \bigcup \{L_1^i \mid i \in \mathbb{N}\} = L_1^+ \cup L_1^0, \end{aligned}$$

where $L_1^0 = \{\lambda\} \neq \lambda$.

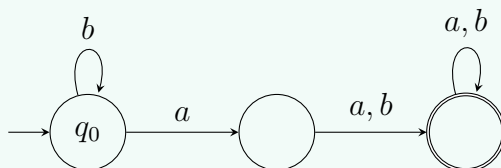
Note: $L_1^* = L_1^+$ iff $\lambda \in L_1$.

If $\lambda \notin L_1$, then $q_1 \notin F_1$.

Idea: Add q_1 to F_1 . (But this doesn't always work.)

Example 5.2.1 (Counterexample for naive Kleene star construction)

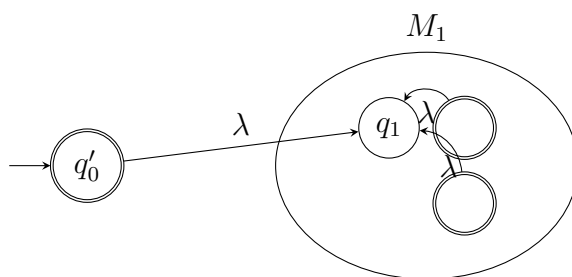
Consider the DFA M with $\Sigma = \{a, b\}$:



Then $aa \in \mathcal{L}(M)$ but $a \notin \mathcal{L}(M_1)$ (where M_1 is the original machine without the modification).

If we naively add q_0 to F , then $aa \in \mathcal{L}(M)$ – correct. But $a \notin \mathcal{L}(M_1)$, yet we might incorrectly accept a .

Correct construction (Idea 2):



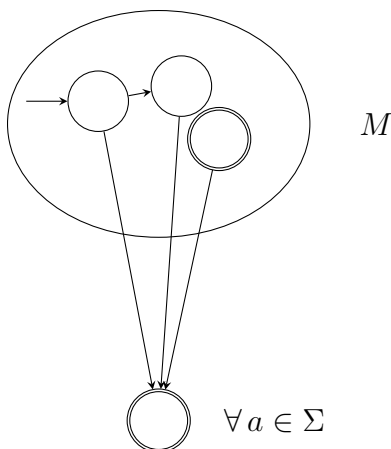
5.2.1 PREFIX Construction

Definition 5.2.1: PREFIX

$\text{PREFIX}(L) = \{x \in \Sigma^* \mid \exists w \in L \text{ s.t. } w \text{ is a prefix of } x\}$, where $L \subseteq \Sigma^*$.

If L is regular, how to show $\text{PREFIX}(L)$ is regular?

Construction: Given a DFA M for L , make every state connect to a (new) final state.



That is: make every state connect to a final state (add λ -transitions from every state to a new accepting state, or equivalently, make every state from which an accepting state is reachable into an accepting state).

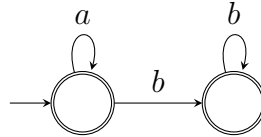
5.3 Second Proof that $T = \{a^m b^n \mid m \neq n\}$ is Not Regular

Lemma 5.3.1

$T = \{a^m b^n \mid m \neq n\}$ is not regular.

Proof #2: To obtain a contradiction, suppose T is regular.

Then $L = \{a\}^* \cdot \{b\}^*$ is regular (since it is recognized by a simple DFA):



So $\mathcal{L}(M) = L$.

Now $(\{a, b\}^* - T)$ is regular (since regular languages are closed under complement).

And $(\{a, b\}^* - T) \cap L = \{a^i b^i \mid i \in \mathbb{N}\}$.

But we proved earlier that this is not regular.

QED.



Chapter 6

Regular Expressions

So far, we have seen two ways to describe regular languages: DFAs and NFAs. In this chapter, we introduce a third, purely algebraic formalism — *regular expressions* — that describes the same class of languages. Regular expressions give a concise, compositional notation for building up languages from simple pieces using union, concatenation, and repetition. The main result of this chapter is that regular expressions describe exactly the regular languages: every regular expression can be converted to a finite automaton and vice versa.

6.1 Definition of Regular Expressions

Definition 6.1.1: Regular Expression

Let Σ be a finite alphabet. The set R of **regular expressions** over Σ is the following inductively defined set of strings:

Base Cases:

- $\phi \in R$ (represents the empty language),
- $\lambda \in R$ (represents the language $\{\lambda\}$ containing only the empty string),
- $a \in R$ for each $a \in \Sigma$ (represents the language $\{a\}$).

Constructor Cases: If $r, s \in R$, then:

- $(r + s) \in R$ (union),
- $(r \cdot s) \in R$ (concatenation),
- $(r^*) \in R$ (Kleene star).

Note:-

By convention, we adopt the following operator precedence to reduce parentheses: Kleene star $*$ binds tightest, then concatenation $\cdot$, then union $+$. For example, $ab^* + c$ is read as $((a \cdot (b^*)) + c)$. We also often omit the $\cdot$ symbol, writing rs instead of $r \cdot s$.

Definition 6.1.2: Language of a Regular Expression

The **language** described (or denoted) by a regular expression $r \in R$ is $L(r)$, defined as a function:

$$L : R \rightarrow \{S \mid S \subseteq \Sigma^*\}$$

Base Cases:

$$L(\phi) = \emptyset$$

$$L(\lambda) = \{\lambda\}$$

$$L(a) = \{a\} \quad \text{for each } a \in \Sigma$$

Constructor Cases:

$$L(r + s) = L(r) \cup L(s)$$

$$L(r \cdot s) = L(r) \cdot L(s) = \{xy \mid x \in L(r) \text{ and } y \in L(s)\}$$

$$L(r^*) = L(r)^* = \bigcup_{i=0}^{\infty} L(r)^i$$

Example 6.1.1 (Regular expressions over $\Sigma = \{a, b\}$)

1. $L(a + b) = \{a\} \cup \{b\} = \{a, b\} = \Sigma$.
2. $L((a + b)^*) = \Sigma^*$, the set of all strings over $\{a, b\}$.
3. $L(a^*b) = \{b, ab, aab, aaab, \dots\} = \{a^n b \mid n \geq 0\}$.
4. $L(a(a + b)^*b) =$ the set of all strings that start with a and end with b of length ≥ 2 .
5. $L(\phi^*) = \{\lambda\}$, since $\emptyset^0 = \{\lambda\}$ and $\emptyset^i = \emptyset$ for all $i \geq 1$.
6. $L(\lambda + aa^*) = \{\lambda\} \cup \{a, aa, aaa, \dots\} = \{a^n \mid n \geq 0\} = L(a^*)$.

6.1.1 Useful Identities

The following identities hold for all regular expressions r, s, t :

$$\begin{array}{ll}
 r + \phi = r & \phi \cdot r = r \cdot \phi = \phi \\
 r + r = r & \lambda \cdot r = r \cdot \lambda = r \\
 r + s = s + r & (r + s) \cdot t = r \cdot t + s \cdot t \\
 (r + s) + t = r + (s + t) & r \cdot (s + t) = r \cdot s + r \cdot t \\
 (r \cdot s) \cdot t = r \cdot (s \cdot t) & r^* = \lambda + r \cdot r^* \\
 \phi^* = \lambda & (r^*)^* = r^*
 \end{array}$$

These can all be verified by showing that both sides denote the same language.

6.1.2 Equivalence of Regular Expressions

Definition 6.1.3: Equivalence of Regular Expressions

Two regular expressions r and s are **equivalent**, written $r \equiv s$, if they denote the same language: $L(r) = L(s)$.

To prove that two regular expressions are equivalent, one proves $L(r) \subseteq L(s)$ and $L(s) \subseteq L(r)$. That is, show that every string matched by r is also matched by s , and vice versa.

Example 6.1.2

We show $r^* \equiv \lambda + r \cdot r^*$.

($\subseteq$) Let $x \in L(r^*) = \bigcup_{i=0}^{\infty} L(r)^i$. If $x \in L(r)^0 = \{\lambda\}$, then $x = \lambda \in L(\lambda)$. Otherwise, $x \in L(r)^i$ for some $i \geq 1$, so $x = x_1 x_2 \cdots x_i$ where each $x_j \in L(r)$. Then $x_1 \in L(r)$ and $x_2 \cdots x_i \in L(r)^{i-1} \subseteq L(r^*)$, so $x \in L(r) \cdot L(r^*) = L(r \cdot r^*)$. In either case, $x \in L(\lambda + r \cdot r^*)$.
 ($\supseteq$) If $x \in L(\lambda)$, then $x = \lambda \in L(r)^0 \subseteq L(r^*)$. If $x \in L(r \cdot r^*)$, then $x = yz$ where $y \in L(r)$ and $z \in L(r^*) = L(r)^i$ for some $i \geq 0$. So $x = yz \in L(r)^{i+1} \subseteq L(r^*)$. $\odot$

6.1.3 Generalized Regular Expressions

Standard regular expressions use only $+$, $\cdot$, and $*$. **Generalized regular expressions** extend the syntax with additional operators. These do not increase the expressive power (since regular languages are closed under these operations), but they can make expressions more concise.

Definition 6.1.4: Generalized Regular Expression

A generalized regular expression extends the syntax of regular expressions with:

$$\begin{aligned}
 r \cap s & \text{ with } L(r \cap s) = L(r) \cap L(s) \quad (\text{intersection}) \\
 r^+ & \text{ with } L(r^+) = L(r)^+ = \bigcup_{i=1}^{\infty} L(r)^i \quad (\text{one or more repetitions}) \\
 r - s & \text{ with } L(r - s) = L(r) - L(s) \quad (\text{set difference}) \\
 \bar{r} & \text{ with } L(\bar{r}) = \Sigma^* - L(r) \quad (\text{complement})
 \end{aligned}$$

Note:-

Note that $r^+ = r \cdot r^*$ and $L(\bar{\phi}) = \Sigma^* - \emptyset = \Sigma^*$. Also, $r^* = \lambda + r^+$, so Kleene star and Kleene plus are interdefinable.

6.2 Regular Expressions and Finite Automata

The central result of this chapter is that regular expressions describe exactly the regular languages. We prove both directions.

6.2.1 From Regular Expressions to Finite Automata

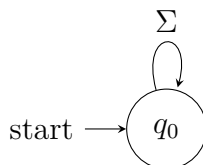
Theorem 6.2.1

For every regular expression r , there is a finite automaton M such that $L(r) = L(M)$.

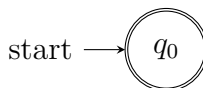
Proof: We proceed by structural induction on the regular expression r .

Base Cases:

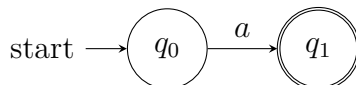
- $r = \phi$: The automaton has a single state q_0 that is not accepting, with all transitions leading back to q_0 . This accepts no strings, so $L(M) = \emptyset = L(\phi)$.



- $r = \lambda$: The automaton has a single state q_0 that is accepting. It accepts only the empty string.



- $r = a$ for some $a \in \Sigma$: The automaton has two states, with a single transition on a from the start state to the accepting state.



Constructor Cases:

Suppose, by the induction hypothesis, that there exist finite automata M_r and M_s such that $L(M_r) = L(r)$ and $L(M_s) = L(s)$.

- $r + s$ (union): Since regular languages are closed under union (Chapter 5), $L(r) \cup L(s)$ is regular. Concretely, we can use the NFA construction for union: create a new start state q_0 with λ -transitions to the start states of M_r and M_s .
- $r \cdot s$ (concatenation): Since regular languages are closed under concatenation (Chapter 5), $L(r) \cdot L(s)$ is regular. Concretely, add λ -transitions from each accepting state of M_r to the start state of M_s , and make only the accepting states of M_s accepting.
- r^* (Kleene star): Since regular languages are closed under Kleene star (Chapter 5), $L(r)^*$ is regular. Concretely, create a new accepting start state q_0 with a λ -transition to the start state of M_r , and add λ -transitions from each accepting state of M_r back to the start state of M_r .

In each constructor case, the resulting NFA recognizes the desired language. Since every NFA can be converted to a DFA, the result follows. ☺

6.2.2 From Finite Automata to Regular Expressions

To prove the converse — that every finite automaton can be converted to an equivalent regular expression — we introduce *general transition graphs* and use the technique of *state elimination*.

6.3 General Transition Graphs

A general transition graph (GTG) is a generalization of an NFA where the edges are labelled by regular expressions instead of single symbols. This serves as an intermediate representation in the conversion from finite automata to regular expressions.

Definition 6.3.1: General Transition Graph (GTG)

A **general transition graph** is a 5-tuple $G = (Q, \Sigma, \delta, q_0, f)$ where:

- Q is a finite set of states,
- Σ is a finite alphabet,
- $q_0 \in Q$ is the start state,
- $f \in Q$ is the unique accept state, with $q_0 \neq f$,
- $\delta : Q \times Q \rightarrow R$ is the transition function, which assigns a regular expression to each ordered pair of states. If there is no intended transition from q to q' , we set $\delta(q, q') = \phi$.

Definition 6.3.2: Language of a GTG

The language of a GTG G is:

$$L(G) = \{x \in \Sigma^* \mid \text{there is a path from } q_0 \text{ to } f \text{ and } x \text{ is in the language described by the concatenation of the labels of the edges on the path}\}$$

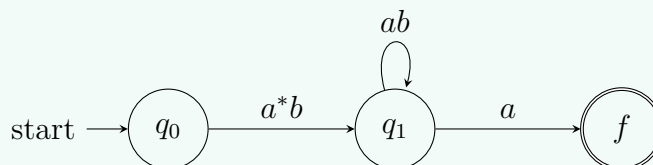
More precisely, a string $x \in \Sigma^*$ is **accepted** by G if and only if there exist $j \in \mathbb{N}$, states $s_0, s_1, \dots, s_j \in Q$ with $s_0 = q_0$ and $s_j = f$, and strings $x_1, x_2, \dots, x_j \in \Sigma^*$ such that:

1. $x = x_1 \cdot x_2 \cdots x_j$,
2. for each $i \in \{1, \dots, j\}$, $x_i \in L(\delta(s_{i-1}, s_i))$.

In other words, x can be decomposed into pieces, each of which is matched by the regular expression labelling the corresponding edge along a path from q_0 to f .

Example 6.3.1 (A simple GTG)

Consider the GTG over $\Sigma = \{a, b\}$ with three states:



This GTG accepts strings of the form: some a 's followed by b , then any number of ab pairs,

then a final a . For example, $ba \in L(G)$ (take zero a 's before the b , zero loops, then a), $aababa \in L(G)$ (aab , then ab , then a). The language is $L(G) = L(a^*b(ab)^*a)$.

6.3.1 NFA to GTG Conversion

The first step in converting a finite automaton to a regular expression is to convert it into a GTG. A GTG requires two structural properties that an NFA may not have:

1. A **single accept state** f (an NFA can have many), and
2. The accept state f has **no outgoing transitions**, and the start state q_0 has **no incoming transitions**.

Additionally, the two formalisms represent transitions differently:

- In an NFA, the transition function is $\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow \mathcal{P}(Q)$. It answers: “from state q , reading symbol a , which states can I reach?”
- In a GTG, the transition function is $r : Q \times Q \rightarrow R$. It answers: “what strings allow me to go directly from state q to state q' ?”

So the conversion must **re-index** the transitions: instead of grouping by (state, symbol) $\rightarrow$ set of target states, we group by (source state, target state) $\rightarrow$ what symbols allow this move.

Theorem 6.3.1

For every NFA M , there is a GTG G such that $L(M) = L(G)$.

Proof: Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA.

Step 1: Create the single accept state. A GTG needs exactly one accept state with no outgoing edges. The NFA may have multiple accepting states $F = \{f_1, f_2, \dots, f_m\}$, and these states may have outgoing transitions. To fix this, introduce a brand new state $f \notin Q$ and set $Q' = Q \cup \{f\}$. The state f will be the sole accept state of the GTG.

Step 2: Create the single start state (if needed). If the start state q_0 has incoming edges in the NFA, introduce a brand new state $q'_0 \notin Q'$, set $Q' = Q' \cup \{q'_0\}$, and use q'_0 as the GTG's start state. (If q_0 has no incoming edges, we can keep it as the start state.)

Step 3: Define the GTG transition function. We now define $r : Q' \times Q' \rightarrow R$. The idea is to look at each pair of states (q, q') and ask: “in the original NFA, what symbols let me move from q to q' in a single step?” and then combine them with $+$ (union).

Transitions involving the new accept state f :

- For each original accepting state $q \in F$: set $r(q, f) = \lambda$. This means “you can move from q to f by reading the empty string.” This captures the fact that q used to be accepting — reaching q in the NFA now leads to f (acceptance) in the GTG.
- For each non-accepting state $q \in Q - F$: set $r(q, f) = \phi$. There is no way to reach f from a non-accepting state, since only the original accepting states should lead to acceptance.

- For all $q \in Q'$: set $r(f, q) = \phi$. The accept state f has no outgoing transitions (this is a structural requirement of GTGs for state elimination to work).

Transitions between original states $q, q' \in Q$: Collect all symbols that allow a move from q to q' in the NFA:

$$S_{q,q'} = \{a \in \Sigma \cup \{\lambda\} \mid q' \in \delta(q, a)\}$$

- If $S_{q,q'} = \{a_1, a_2, \dots, a_k\}$ is non-empty, set $r(q, q') = a_1 + a_2 + \dots + a_k$.

For example, if the NFA has both an a -transition and a b -transition from q to q' , then $S_{q,q'} = \{a, b\}$ and we set $r(q, q') = a + b$. The GTG edge from q to q' is labelled by the regular expression $a + b$, which matches exactly the strings a and b — precisely the symbols that allowed this move in the NFA.

- If $S_{q,q'} = \emptyset$ (no transition from q to q' in the NFA on any symbol), set $r(q, q') = \phi$. The ϕ label means “no string can traverse this edge,” which is equivalent to having no edge at all.

Transitions involving the new start state q'_0 (if added): set $r(q'_0, q_0) = \lambda$ and $r(q'_0, q') = \phi$ for all other $q' \neq q_0$.

Correctness. The resulting GTG accepts the same language as M :

- ($L(M) \subseteq L(G)$): Let $x \in L(M)$, so there is an accepting computation of M on x that ends in some state $q_f \in F$. Each step of this computation follows a transition $\delta(q, a) \ni q'$, and by construction the GTG has a as a summand in $r(q, q')$. So the same sequence of states forms a path in G from q_0 to q_f , and the labels match x . Appending the final edge from q_f to f (labelled λ) gives a path from q_0 to f in G matching $x \cdot \lambda = x$. Hence $x \in L(G)$.
- ($L(G) \subseteq L(M)$): Let $x \in L(G)$, so there is a path in G from q_0 to f matching x . The only way to reach f is from some state $q_f \in F$ (since $r(q, f) = \phi$ for $q \notin F$). The path from q_0 to q_f uses only edges between original states, and each edge label $a_1 + \dots + a_k$ means each piece of x along that edge is some $a_i \in S_{q,q'}$ — which corresponds to a valid NFA transition. So the path from q_0 to q_f is a valid accepting computation of M on x . Hence $x \in L(M)$.

☺

Note:-

The self-loop case deserves special attention. If the NFA has transitions $\delta(q, a) \ni q$ and $\delta(q, b) \ni q$, then $S_{q,q} = \{a, b\}$ and the GTG gets a self-loop on q labelled $a + b$. This is correct: traversing the self-loop once matches one character that is either a or b , just like the NFA.

6.3.2 GTG to Regular Expression (State Elimination)

The key idea of state elimination is: given a GTG with n states, pick any state q_x that is neither the start state nor the accept state, and remove it. All paths that used to pass through q_x are “absorbed” into the labels of the remaining edges. This produces a GTG with $n - 1$ states that accepts the same language. Repeating until only q_0 and f remain, we can read off the regular expression directly.

Theorem 6.3.2

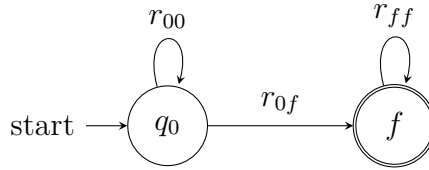
For every GTG G , there is a regular expression r such that $L(G) = L(r)$.

Proof: By induction on the number n of states in G .

Let $P(n)$ = “for all generalized transition graphs G with n states, there exists a regular expression r such that $L(r) = L(G)$.”

Base Case: $n = 2$.

Consider an arbitrary GTG G with 2 states q_0 and f . The only possible transitions are: a self-loop on q_0 (labelled r_{00}), an edge from q_0 to f (labelled r_{0f}), and a self-loop on f (labelled r_{ff}). Note that there is no edge from f to q_0 since f has no outgoing transitions (other than a self-loop).



Any accepting path must: loop on q_0 some number of times, traverse the edge to f , then loop on f some number of times. Therefore:

$$L(G) = L(r_{00}^* \cdot r_{0f} \cdot r_{ff}^*)$$

This is a regular expression, so $P(2)$ is true.

Induction Step: Let $n \geq 3$ and assume $P(n-1)$.

Let $G = (Q, \Sigma, \delta, q_0, f)$ be an arbitrary GTG with n states. Since $n \geq 3$, there exists at least one state $q_x \in Q - \{q_0, f\}$.

Construct a new GTG $G' = (Q', \Sigma, \delta', q_0, f)$ where:

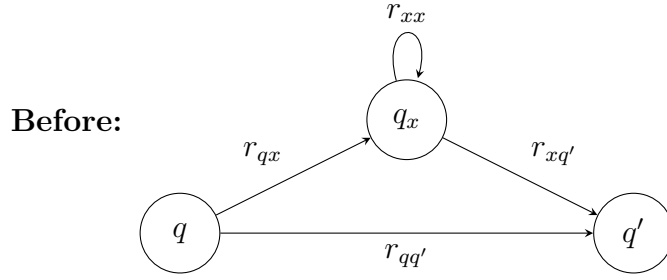
$$Q' = Q - \{q_x\}$$

and for all $q, q' \in Q'$:

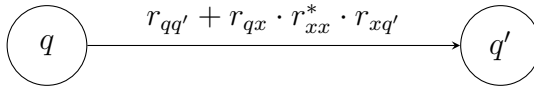
$$\delta'(q, q') = \delta(q, q') + \delta(q, q_x) \cdot \delta(q_x, q_x)^* \cdot \delta(q_x, q')$$

The intuition is: the new label $\delta'(q, q')$ accounts for two ways to get from q to q' :

1. Go directly from q to q' without passing through q_x (the $\delta(q, q')$ part).
2. Go from q to q_x , loop on q_x any number of times, then go from q_x to q' (the $\delta(q, q_x) \cdot \delta(q_x, q_x)^* \cdot \delta(q_x, q')$ part).



After:



Claim: $L(G) = L(G')$.

More precisely, for all $q, q' \in Q'$ and all $x \in \Sigma^*$: there is a path from q to q' in G (possibly passing through q_x) whose concatenated labels match x if and only if there is a path from q to q' in G' whose concatenated labels match x .

In particular, taking $q = q_0$ and $q' = f$, it follows that $L(G) = L(G')$.

Since G' has $n - 1$ states, by the induction hypothesis $P(n - 1)$, there exists a regular expression r such that $L(G') = L(r)$. Hence $L(r) = L(G)$.

By generalization (since G was an arbitrary GTG with n states), $P(n)$ holds.

By induction, $P(n)$ is true for all $n \geq 2$. ☺

Combining the NFA-to-GTG conversion with the GTG-to-regular-expression conversion, we obtain:

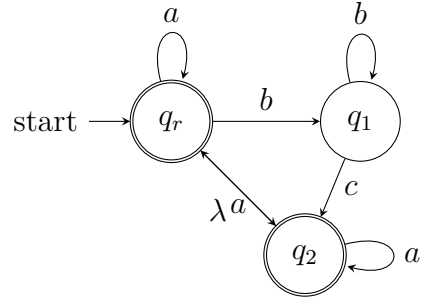
Theorem 6.3.3 Kleene's Theorem

A language $L \subseteq \Sigma^*$ is regular if and only if there exists a regular expression r such that $L = L(r)$.

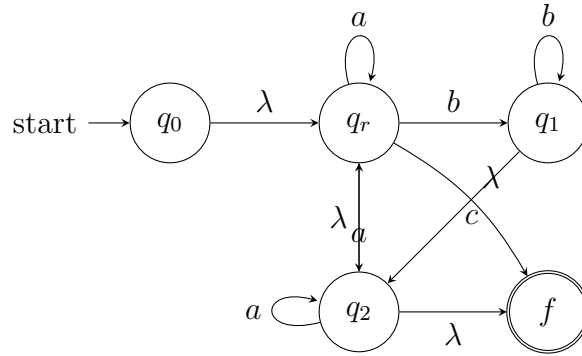
6.3.3 Example: NFA to Regular Expression via State Elimination

We now walk through a complete example of converting an NFA to a regular expression using state elimination.

Consider the NFA M over $\Sigma = \{a, b, c\}$ with states q_r (start, accepting), q_1 , and q_2 (accepting):



Step 1: Convert to GTG. Add a new start state q_0 with a λ -transition to q_r , and a new accept state f with λ -transitions from each original accepting state (q_r and q_2):



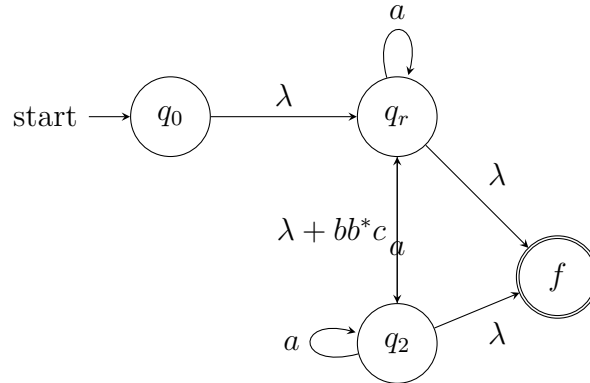
This GTG has 5 states: q_0, q_r, q_1, q_2, f .

Step 2: Eliminate q_1 . The state q_1 has:

- Incoming edge from q_r : label b
- Self-loop: label b
- Outgoing edge to q_2 : label c

Using the state elimination formula, the new edge from q_r to q_2 absorbs paths through q_1 :

$$\delta'(q_r, q_2) = \lambda + b \cdot b^* \cdot c = \lambda + bb^*c$$



Step 3: Eliminate q_r . After absorbing all paths through q_r :

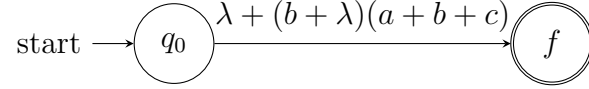
$$\delta'(q_0, q_2) = \phi + \lambda \cdot a^* \cdot (\lambda + bb^*c) = a^*(\lambda + bb^*c)$$

$$\delta'(q_0, f) = \phi + \lambda \cdot a^* \cdot \lambda = a^*$$

$$\delta'(q_2, q_2) = a + a \cdot a^* \cdot (\lambda + bb^*c)$$

$$\delta'(q_2, f) = \lambda + a \cdot a^* \cdot \lambda = \lambda + aa^*$$

Step 4: Eliminate q_2 . After absorbing all paths through q_2 , we are left with:



Therefore:

$$L(G) = L(\lambda + (b + \lambda)(a + b + c))$$

This is a regular expression equivalent to the original NFA M .